

Blockchain-based application for the traceability of complex assembly structures

Marlene Kuhn ^{*}, Felix Funk, Guanlai Zhang, Jörg Franke

Institute for Factory Automation and Production Systems, Friedrich-Alexander University Erlangen-Nuremberg, Fuerther Str. 246b, Nuremberg, 90429, Germany

ARTICLE INFO

Keywords:

Traceability
dApp
Blockchain
ERC 1155
Assembly
Smart manufacturing
Graph model

ABSTRACT

Production processes are becoming increasingly complex and decentralized, spanning across multiple manufacturing areas, factories, and even countries. Thereby is the development of a holistic and data-continuous traceability system, which provides transparency of records for the entire manufacturing flow from raw materials to the final product, one of the main challenges. While some solutions already address traceability issues in supply chain management and manufacturing, customized and volatile industries, in which each product features a unique configuration, are still lacking appropriate models and proofs of concept. In this research, we develop a decentralized blockchain application called TokenTrail, which focuses on the specific traceability requirements of multi-hierarchical assembly structures. The proposed blockchain architecture is based on a consortium Ethereum network and a Proof of Authority consensus, which delivers a trusted and shared database within an economic processing framework. To overcome the challenges of managing liaison transformations, we develop an assembly token manager based on the semi-fungible ERC 1155 token, which enables a direct representation of complex assembly processes and structures containing unique parts and batches within one smart contract. A user-friendly front-end application simplifies the querying of the recorded traceability data and visualizes existing dependencies.

1. Introduction

In discrete manufacturing, distinct and identifiable items are produced by joining parts and components into assemblies of higher hierarchy levels [1]. With the rise of mass-customization and production decentralization, discrete assembly processes are becoming increasingly complex, which manifests itself in growing structural part complexities paired with dynamically changing product configurations [2, 3]. Moreover, the trend towards assembly customization and production in lot-size one, such as in the automotive, mobility, or aerospace industry, leads to ever-changing product configurations and therefore generates unique data histories for each product [4]. Product complexity is further amplified through the continuous elimination of purely mechanical components and the increasing proportion of mechatronic parts, consisting of mechanic, electric, and software functions [5]. Mechatronic products often consist of thousands of distinct and heterogeneous parts [4] and therefore require a wide range of manufacturing technologies. Such products are assembled in increasingly decentralized manufacturing processes [6], often in

geographically and organizationally distributed supply chains. The resulting segmentation and decentralization of production chains exacerbate their information management. Consequently, interoperability, as well as data integration and exchange, have become a major challenge in discrete assembly processes and induce the need for innovative traceability solutions [3,7].

Although no completely unambiguous definition of traceability exists in literature, many authors refer to traceability as a data collection, storage, and reporting process for identifiable products and defined operations [8,9]. Hence, a traceability system aims at connecting quality-relevant information across heterogeneous data input domains and augmenting manufacturing transformations to a most accurate and holistic digital representation. It, therefore, provides a macro data perspective on the manufacturing flow, which can be used to identify errors timely, manage recalls and liability claims, as well as improve processes proactively based on a connected data set. Especially in the field of supply chain management (SCM), the importance of following product composition over all stages of manufacturing and assembly is stressed [10]. Each object's state needs to be monitored

^{*} Corresponding author.

E-mail address: marlene.kuhn@faps.fau.de (M. Kuhn).

<https://doi.org/10.1016/j.jmansys.2021.04.013>

Received 17 December 2020; Received in revised form 6 April 2021; Accepted 20 April 2021

Available online 29 April 2021

0278-6125/© 2021 The Society of Manufacturing Engineers. Published by Elsevier Ltd. All rights reserved.

continuously by logging location and controller data, association data with other objects, and object properties [11]. So-called traceability systems are employed for this task. While the motivation for using these systems depends on the use-case, common goals are the reduction of recall risk and costs, rising documentation and transparency requirements, as well as the necessity of fighting counterfeits or managing aggravated product complexity [9,12–14]. Especially in smart manufacturing industries, the emphasis lies on collecting data of product configuration structures and monitoring relationships between subordinate and super-ordinate material lots [8,15]. The need for implementing traceability has significantly risen for smart manufacturing environments, while at the same time the benefits of a traceability system simultaneously grow with the increasing amount of data and connected devices [16].

Traceability systems rely on an appropriate information system infrastructure to collect and safely maintain traceability records across different manufacturing domains [16]. The information system determines how traceability-relevant objects and their properties are virtually represented as well as to which degree and granularity object transformations can be digitally monitored. Furthermore, it defines the aggregation and storage structure and sets the boundaries of how the data can be queried and visualized [4]. A central part of each information system is a database, which holds the traceability data across all contributing organizational units. Within the last couple of years, blockchain has emerged as an agglomerated system of information technologies and protocols, which enable promising features for traceability applications, such as maintaining a trusted and immutable ledger of data records and transactions in a peer-to-peer environment. Based on the primary work of Satoshi Nakamoto (Bitcoin) [17], further applications beyond cryptocurrencies, such as logistics, trade, digital identity management, or energy management, emerged. Within this development, a wide range of blockchain-based traceability systems have been proposed. Early concepts highlighted the technology's suitability to facilitate higher supply chain transparency and data integrity and drafted first models for digital agent management and blockchain-based data entry across supply chain units [18,19]. In following works, smart contracts were increasingly used to represent traceability functions, such as receiving or transferring products [20], which allowed a closer connection between physical processes and virtual transactions. Later, the focus on the digital representation of physical assets was further enhanced, which can be considered as a starting point for the traceability of manufacturing processes [21]. Simultaneously, the idea of combining on- and off-chain data storage arose, which allowed performance improvements by focusing on application-relevant data within the blockchain network [21,22]. In this regard, concepts for smart manufacturing service collaboration, lifecycle data, and blockchain-enabled digital twins were developed [23,24].

Most of the previously proposed blockchain-based traceability solutions integrate object or asset data as primary data, events, or even off-chain data. However, physical assets could also be represented using so-called blockchain tokens [25], which would not only enhance the functionalities within blockchain-based applications but also allow a closer mapping of physical object behavior and digital representation. In Ethereum, one of the most popular blockchain ecosystems for smart contract development [26], ERC 20 (Ethereum Request for Comment) is a standard token protocol that describes the rule set for the management of digital assets, which are identical and of the same value [27]. Such fungible tokens act as interchangeable virtual entities and can therefore be used in cryptocurrencies or asset certification. In SCM traceability, the usage of ERC 721 tokens [28] has been proposed to manage non-fungible, unique assets, such as inter-company shipments. Meyer et al. [29] propose a decentralized logistic concept based on the ERC 721 token for uniquely identifiable logistic containers. A smart contract concept is developed for creating, transferring, and approving the tokens as well as monitoring the state changes across different logistic nodes

[29]. While their solution works well for unique and self-contained assets, complex product structures could not be represented based on the ERC 721 framework. The tokenization of parts and products has furthermore been focused in [30], who also propose tracking products and ownership based on an ERC 721 token smart contract. Moreover, in [31], the authors describe part representation through blockchain tokens within the IBM Hyperledger framework. Their application features a web client as well as a mobile client and covers some traceability functions such as registering or verifying the authenticity of parts. However, no option to replicate processes based on batch logic such as merging parts is provided. Westerkamp et al. [32,33] address this issue by developing a blockchain-based supply chain traceability system that enables documenting assembly processes and thus tracing part relations and compositions. In this traceability system, product batches are represented as unique data structures that contain information about the product such as the materials used for its production. For each kind of product, a data structure is defined in a separate smart contract. Each batch of a product is subsequently a token in the corresponding contract. Tokens can be transferred between companies in the supply chain, and manufacturing processes are defined in so-called recipes that specify the input products necessary to create a specific output product. The proposed traceability system, which is validated for an example implementation in the wood processing industry, offers an intriguing approach by introducing batch logic and utilizing event logs as a cheap form of data storage. However, it is ill-suited to use cases in which product compositions are dynamic and new types of products are frequently introduced, as both require developing and deploying new smart contracts, which strongly limits the system's scalability. It is therefore not applicable in customized industries with a high product variety and assembly complexity.

The general applicability, characteristics, and benefits of blockchain for traceability purposes have been demonstrated in various works [19–21,29,34]. While more recent research suggests, that the representation of products as blockchain tokens is promising [29,33], current literature does not provide a concept or solution, which allows to holistically virtualize and trace multi-hierarchical and dynamically configured assembly structures across decentralized supply chains. Moreover, the proposed traceability blockchain architecture needs to be economic and sufficiently scalable for this type of use case. In this context, there is a general shortage of fully functional prototypes in blockchain research to demonstrate the applicability of proposed concepts through credible data. In this paper, we close this research gap by providing the following contributions:

- We develop a decentralized traceability application called **Token-Trail**, which enables tracing complex assembly structures consisting of unique parts and batches within one contract. The smart contract behind TokenTrail operates as a general-purpose assembly token manager and can be instantiated for any type of assembly in any type of industry, making our solution transferable to a large number of use cases.
- We propose a blockchain architecture that allows supplier integration and demonstrates its applicability for the monitoring of mass-customized manufacturing through an economic design and consensus protocol.
- We demonstrate the solution's performance and scalability through the fully functional software product prototype TokenTrail. Reliable testing and simulation data for a realistic automotive assembly use case validate the developed concept.

The remainder of this paper is structured as follows: First, we provide the theoretical background on the manufacturing logic of multi-hierarchical and customized assemblies. We further deduce the resulting traceability challenges focusing on batch and unique part liaisons as well as cross-component relationship tracking. We propose an ERC-1155-based traceability concept, which enables tracing assembly

processes using fungible and non-fungible tokens within one smart contract. The blockchain architecture is then specified by focusing on consensus design, data governance, and event logic. To implement the proposed concept in a user-friendly manner, we develop the decentralized application (dApp) **TokenTrail**, which allows the user to recreate assembly processes with tokens and to retrieve information about current object states as well as past processes. Our system is then validated and its performance is evaluated for an automotive assembly process. We base our solution on our early conceptual works [34] and primary proof of concepts [35]. To the best of our knowledge, this is the first approach, which demonstrates a functional and performant blockchain application enabled by efficient token management for the traceability of complex assembly structures.

2. Manufacturing traceability of complex assembly structures

2.1. Assembly transformation processes

This section lays the foundation for the traceability systematic of our token-based blockchain application by giving the theoretical background on assembly transformation processes and their resulting traceability challenges. An assembly can be characterized as a geometric and topological product liaison, which is defined by its inter-part relationships and its precedence or sequence constraints [36]. Assemblies have hierarchy levels and consist of sub-modules, parts, and sometimes basic or raw materials [5,37]. Each sub-component of an assembly is characterized by unique features or attributes, while the relationship between those parts defines the joining logic and sequence [5]. In a customer-oriented manufacturing concept, each final part or product composes a specifically configured multi-level product structure with unique attributes [37], in which all parts or sub-modules are produced in a specified lot-size on defined capacities or resources [38]. The representation of complex, multi-hierarchical assemblies is often accomplished using graphs, in which the nodes represent the parts and edges their relationships [36,39]. An exemplary acyclic directed graph-based representation of an assembly structure is depicted in Fig. 1. Each node of the assembly structure hierarchy embodies a material, part, or sub-module according to the product's structured assembly bill of material (BoM). For traceability purposes, components are identified either as batches (if lot size $n > 1$), visualized by white nodes, and referenced by a unique batch identifier (UBID), or unique parts (if lot size = 1), visualized by grey nodes and identified by a unique item identifier (UIID). Modules of higher hierarchies integrate those batches and unique parts, whereby batches can be split and shared by different sub-modules in the product structure, e.g. a sub-module A consists of 5 components of batch B and 2 components of batch C while

sub-module X contains a unique part Y and 3 parts of batch B.

While the features of the components are described through static attribute-value pairs av (e.g. color, serial number, tolerances, etc.), dynamic component features (e.g. quality state, approval state, processing state, etc.) are described through states s , whose changes are triggered through time-sensitive manufacturing events e . To manufacture the defined assembly structure, sequenced part transformations and state updates need to take place, as shown in Fig. 2. Next-level hierarchies are generated by joining processes (e.g. p_1 or p_3 in Fig. 2), which generate a directed connection between a number of input components and a number of output components. Within the transformation process p_1 , a specified amount of batch B_i (consisting of identical elements i) is joined with a specified amount of batch B_j (consisting of identical elements j) to form a uniquely identified component C_i . Consequently, subsets of both batches then compose C_i , as shown in (1).

$$B_i = \{i, i, \dots, i\}, B_i^c \subseteq B_i \quad \text{and} \quad B_j = \{j, j, \dots, j\}, B_j^c \subseteq B_j \quad (1)$$

$$C_i = B_i^c \cup B_j^c$$

In the next process p_2 , the component undergoes a quality test and receives a quality state $s(C_i)$. In p_3 , the subassembly C_i is joined with a set amount from batch B_k to form the next-level hierarchy component C_j . As shown in Fig. 2, each transformation or status update is performed by a resource (e.g. r_1 , r_2 , and r_3), which takes over the controller function of the product.

The idea of location- and resource-bound part control with its induced need for monitoring resource and control changes emerged in early works [40] and has become an integral part of traceability and data management concepts. As resources can be machines, operators, or work stations, such as logistics or quality checkpoints, we refer to them as traceability-relevant actors (TRA) in the following sections. Accordingly, for a holistic traceability picture of complex assembly manufacturing, the system needs to encompass the product structure data including each's parts identification number (UIID or UBID) and quantity (1 to n), process transformation logic represented through state-changes, as well as affiliation to and control shifts by TRA. The integration of those data domains is often referred to as PPR (product, process, resource) data management and sets high demands on the object representation within each domain and the connection logic between the domains [39,41,42].

2.2. Traceability challenges

Conventional manufacturing traceability solutions often apply a centralized database system with appropriate traceability querying functions, which can be called and visualized through a user interface

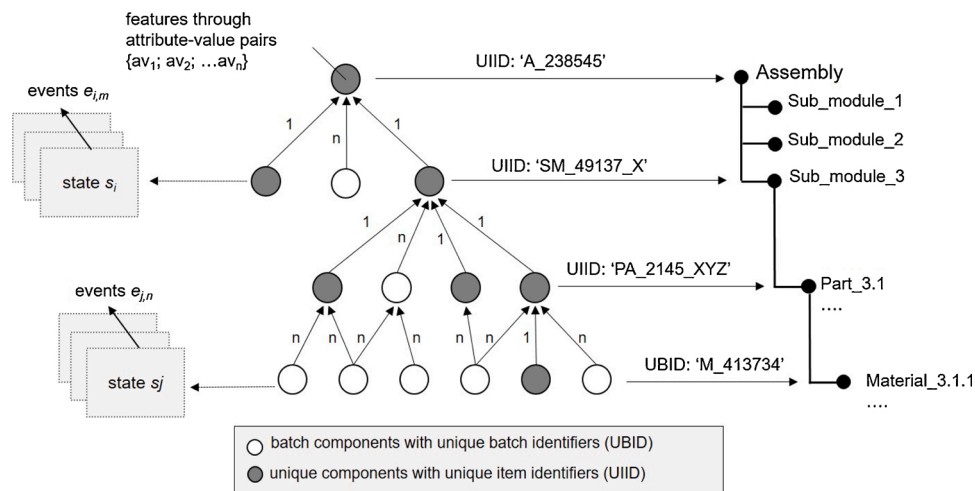


Fig. 1. Graph-based representation of an assembly structure consisting of batches and unique parts.

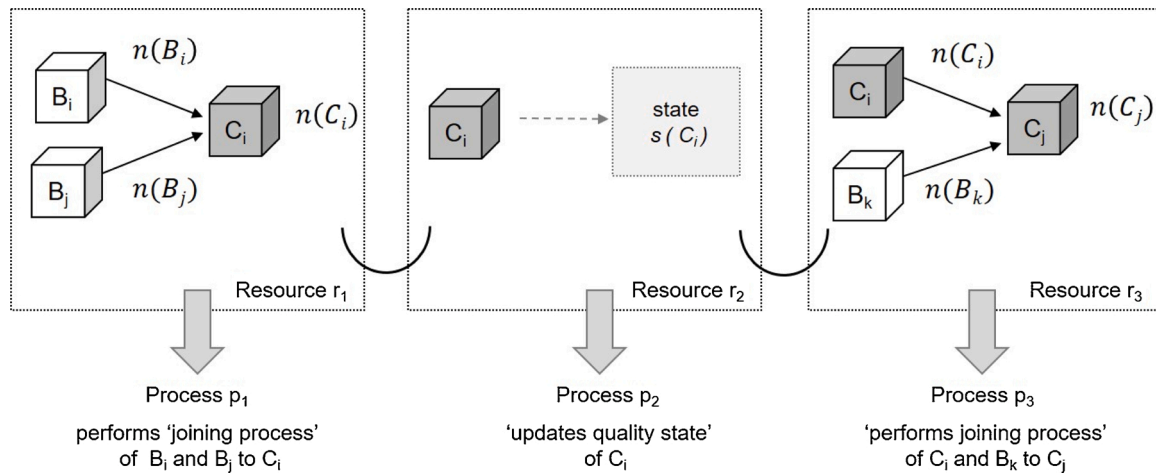


Fig. 2. Transformation logic of batches and components in assembly processes.

(UI) [11,43]. These databases are commonly hosted locally (e.g. by the most powerful participant in the supply network like the OEM [44,45]) or by an intermediary firm (e.g. a cloud provider). While centralized traceability databases offer some advantages, such as a general high technological maturity as well as a wide selection of compatible manufacturing software, they also come with some limitations. Sending sensitive data to a central entity not only places high demands on trust but also generates fees and transaction costs due to the additional intermediary. The issue of trust is particularly relevant, as the traceability database functions as a risk management tool and the source of truth in the case of a recall or liability claim. In this context, changing organizational structures towards higher integration with vertical shifts of value-added tasks, have led to distributed ledger technologies (DLT) gaining in popularity. With the advancement of DLT, the sovereignty over the traceability-relevant process, product, and resource data lies with all participants of the value generation network, as shown in Fig. 3. In the distributed structure, all companies contributing to the final product thus share risk, costs, and data sovereignty.

However, due to the novelty of blockchain and DLT, the holistic traceability of PPR-data for complex assembly structures remains a great challenge. Product assembly transformations need to be represented by relevant attributes and linked with state-sensitive process data as well as resource and controller data [39] throughout a possibly decentralized manufacturing flow. Consequently, the digital model needs to reproduce the functions or

actions of the real manufacturing systems with proper linkages between the digital and real-world systems via logical and physical models and interaction mechanisms [46]. The goal should be a realistic representation of all input-output relations and transformation processes of both, batches and unique parts. Several researchers have highlighted the need for tracing more complex assemblies in distributed manufacturing through blockchain, such as mapping entire product structures to a blockchain-based digital twin model [24]. While blockchain-based traceability solutions for macro data of rigid (non-complex) objects, such as order or package tracking through location changes and owner changes, are comparably mature (e.g. [21,29,31]), tracing complex products still lacks suitable proof of concepts.

3. Research design and requirement framework

To fill the identified research gap, we develop a lean blockchain architecture and decentralized application, which allow tracing complex and dynamically changing assembly structures. As a means to overcome the derived shortcomings of available traceability proposals, we deduce design principles (DP) and technical specifications (TS) for our solution in alignment with the methodology for manufacturing software development presented in [47].

- DP1: Ability to trace multi-level assembly structures holistically in a decentralized network structure.

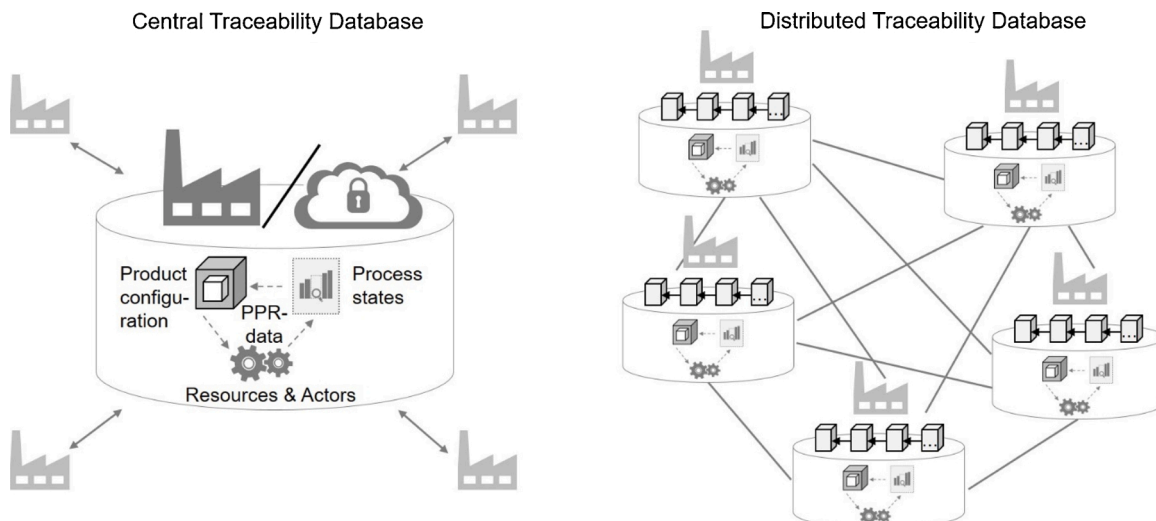


Fig. 3. Comparison of centralized and distributed storage for traceability-relevant PPR-data.

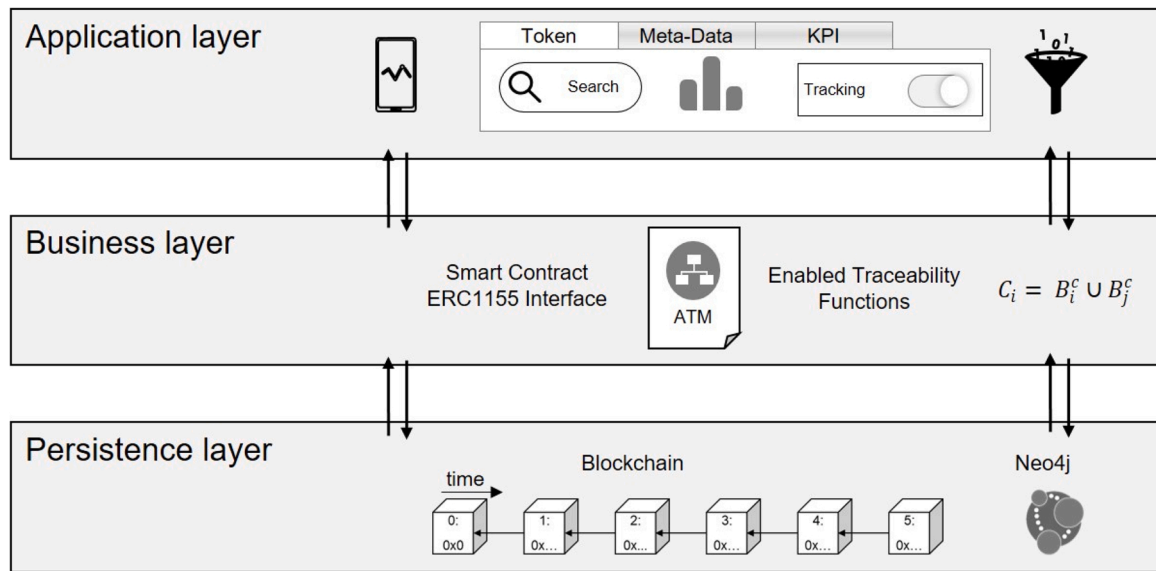


Fig. 4. Architectural layers of the blockchain-based traceability solution.

- DP2: Ability to map any number of input consumption of batches and unique parts to any number of output emittance of batches or unique parts.
- DP3: Ability to assign and monitor control over parts, components, and sub-modules and to transfer parts between different controller entities.
- DP4: Ability to update and monitor static attributes of parts as well as dynamic state changes over the manufacturing flow.
- DP5: Ability to maintain and monitor PPR-linkages, such as the connection of certain transformation processes and part configurations with resource data.
- DP6: General ability to provide a traceability system of trust and equal power in a distributed and dynamic manufacturing environment.

Accordingly, to translate the design principles to an appropriate blockchain solution, technical specifications have to be set. The technical specifications are based on our first proofs of concept of this idea [34,35] and extend the derived shortcomings of related works.

- TS1: The solution should allow a direct representation of product structures in the blockchain, which can be retrieved by direct and efficient queries. It needs to efficiently integrate the main traceability-relevant data, which were identified as product structures including identifiers and quantity, TRAs and shifting control, as well as status changes over the manufacturing flow.
- TS2: The application should be based on a general-purpose smart contract design with unlimited instantiation and configuration options. To integrate dynamic product changes, we refrain from built-in object logic in the contract's configuration (e.g. built-in product structures).
- TS3: A lean blockchain architecture with an energy-efficient consensus mechanism ensures parity among OEMs and suppliers, democratically distributes data sovereignty throughout the supply chain and saves energy resources. The blockchain architecture's performance is equivalent to the manufacturing speed and volume of a mass-customized process flow.
- TS4: Interfaces to off-chain data storage systems allow for seamless integration into the manufacturing IT ecosystem.
- TS5: The interaction with the application, such as writing and querying traceability-relevant data is realized through defined operations in an intuitive user interface. Technical information remains

in the application's backend and user-relevant information is provided from a manufacturing and traceability point of view (e.g. machine names instead of account addresses, manufacturing status instead of blockchain events, etc.).

4. Blockchain-based traceability solution for complex assemblies

To describe the different aspects of our traceability solution, we distinguish between three architecture layers, as shown in Fig. 4. The persistence layer describes the data management and processing framework, which is realized by the Ethereum-based permissioned blockchain and proof of authority consensus algorithm. As off-chain storage, we integrate the graph database Neo4j for its benefits in maintaining connected data. The business layer encompasses the functions and logic within the developed smart contract, which mirrors traceability-relevant business processes such as creating parts from batches or transferring parts between resources. The application layer retrieves and translates the data through defined functions and includes a user-friendly front end, which allows users to pass queries and generate result visualizations.

4.1. Persistence layer via Proof of Authority blockchain

A central consideration in regards to the blockchain is the appropriate choice of the consensus mechanism and its structural configuration. For the case at hand, technical specification TS3 (energy-efficiency and democratic data sovereignty) and design principle DP6 (trust establishment in a distributed environment) are especially important, as they address the performance of the application, requirements towards transparency as well as the envisioned degree of participation for all companies within the supply chain. Accordingly, a permissioned blockchain setup based on Ethereum is chosen to guarantee that traceability data is only available to stakeholders in the system. The decision for Ethereum is based on the high maturity and widespread use of its development environment, the possibility to freely develop own

Table 1

Building blocks for implementation based on ERC 1155 and Proof of Authority.

ERC-1155 framework	PoA consensus protocols	Client
Ethereum	Clique, Aura	Geth, Trinity, Aleth
Hyperledger Quorum	Clique, IBTF 2.0, RAFT	Hyperledger Besu, GoQuorum

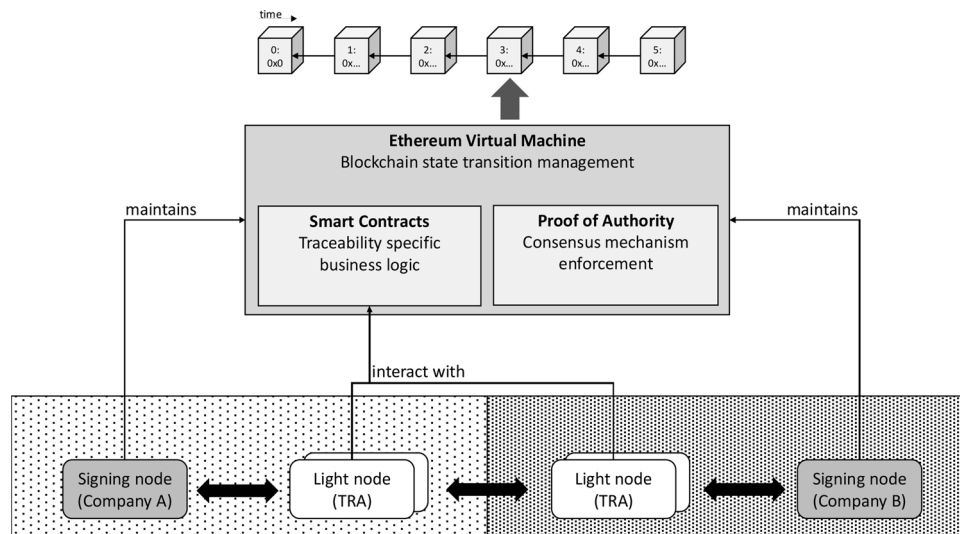


Fig. 5. Schematic relations of blockchain system components and distribution of nodes in the supply chain.

applications on top of the Ethereum framework, as well as its proven applicability for the traceability use case (e.g. as shown in [20,21,33,34]). Furthermore, the now available and here targeted ERC 1155 standard [48], which promises to overcome the multiple inputs to multiple outputs transformation problem of the traceability use case, is native to the Ethereum environment.

As a consensus mechanism, Proof of Authority (PoA) appears as the most appropriate with regard to the aforementioned specifications and design principles. PoA describes a group of byzantine fault-tolerant consensus mechanisms applicable for permissioned blockchains [49]. In these mechanisms, each block is signed by an authorized signing node, which is controlled by a known entity and is randomly chosen out of a set of such nodes [50]. Hence, 51 % of authorized signing nodes would have to conspire to effectively alter the blockchain ex-post. Signers are thus incentivized to act truthfully, as their reputation would suffer upon committing fraud, or, in a business context, they would be liable for the manipulation. This way, the artificially created energy cost for mining that incentivizes truthfulness in Proof of Work (PoW) can be avoided. The reduction in the computational effort also allows for a higher transaction rate while reducing hardware requirements and energy costs as well. PoA does not inhibit the adaptability of the consensus mechanism either, as authority nodes can be elected and dropped through democratic voting by the signing nodes. In comparison with other energy-efficient consensus, such as PBFT, PoA provides better performance and requires fewer message exchange processes [51].

PoA, however, describes a general concept, and details depend on the actual protocol and its integration. For the Ethereum framework, the PoA protocols Clique and Aura can be implemented, while for other frameworks like Hyperledger Quorum based on Ethereum, implementations such as IBFT 2.0 or RAFT are available. As this research is based on Ethereum and the ERC 1155 standard [48], we implement the PoA consensus in the Go Ethereum (Geth) client [52] using the Clique algorithm [50]. A benchmark analysis conducted by De Angelis et al. [49] has shown, that Clique performs well with regard to the CAP theorem (consistency, availability, and partition tolerance), which states that any distributed data basis can guarantee at most two properties of the theorem. In this regard, Clique has demonstrated to achieve availability (A) and partition tolerance (P), whereby eventual consistency (C) can be reached, as forks are ultimately solved through the GHOST protocol [49]. While other clients that support Clique (e.g. Aleth, Trinity, Quorum Besu) should be equally applicable, Geth was chosen as it is commonly used in academia and industry and has been thoroughly investigated in performance benchmarks [53]. Table 1 displays a

comparison of possible frameworks, protocols, and clients that support the defined concept, whereby the chosen framework (Ethereum), PoA protocol (Clique), and client (Geth) are highlighted in bold.

Regarding the integration of the Clique protocol in the traceability context, a uniform distribution of signing nodes across all companies in a supply chain is preferable. In our solution specifically, each company controls a single signing node, so that no centralization of signing power and thus data sovereignty is possible unless the majority of companies votes in favor of it. The necessity for known identities of signing nodes in PoA does not constitute a limitation to this system, as the identities of the companies involved in the supply chain are known already. In

Table 2

Visualization of an exemplary balance mapping for ERC 1155.

Token ID	TRA Accounts				Description
	0 × 1	0 × 2	0 × 3	0 × 4	
1	5	0	0	0	A fungible token, whose 5 instances are owned by one account.
2	1	2	1	1	A fungible token, whose 5 instances are distributed across multiple accounts.
3	1	0	0	0	An effectively non-fungible token, whose single instance is owned by one account.
4	0	0	0	0	A burnt token, which does not have any live instances but is kept for traceability purposes.

Table 3

Data storage overview.

Traceability term	Implementation term	On-/off-chain	Storage	Format
Part location	balance	on-chain	within ATM	public variable
State	status	on-chain	in event logs	json
UBID/UIID (av)	serial number	on-chain	in event logs	json
Part affiliation	controller	on-chain	ATM/ events	public variable/json
Further av	–	off-chain	separate file	free
Link to av	URI	on-chain	in event logs	json

addition to the signing nodes, each traceability relevant actor (TRA) in the supply chain, e.g. machines, quality control stations, or logistic hubs, runs a light node (Geth light client) that does not sign blocks but instead only verifies their contents by downloading the blockchain's headers, which contain the Merkle tree roots. Merkle tree roots in Ethereum are unique fingerprints of all transactions, states, and receipts of a block [54]. These data fingerprints serve as a reference to which each light node can calculate and verify any transaction, account balance, or event without needing to operate the full blockchain. The architecture of the proposed solution is visualized in Fig. 5.

4.2. Business layer via ERC-1155-based Assembly Token Manager

The rules and functionalities defined via the design principles form the business logic of the traceability system and are realized as a smart contract, which we named Assembly Token Manager (ATM) and which we deployed to our Ethereum blockchain. For the tokenized approach, the ERC 1155 token standard as published in [48] is chosen as a foundation and the interface contracts specified in the standard's reference implementation are adopted. The central innovation of the ERC 1155 standard is its efficient integration of fungible and non-fungible tokens, usually realized using ERC 20 and ERC 721 respectively, into a single contract to fulfill TS1 (direct representation of product structures) and TS2 (general-purpose smart contract design). This is achieved through the extension of the two-dimensional balance tracking declared in the other standards by a third dimension. In Solidity, this enhancement reads as follows:

```
mapping (uint256 => mapping(address => uint256)) internal balances;
```

The outer layer maps a token ID (256-bit unsigned integer), which is incremented for each new token, to a map of accounts (address) with balances (256-bit unsigned integer). This double mapping structure can be visualized as a table of token balances. A simple example that demonstrates its capabilities is shown in Table 2. The table contains four distinct token configurations. For clarity, we will from now on refer to the combination of a token ID, its associated information, and the state as a “token”, as opposed to an “instance of a token”, which denominates a single instance of that token in possession of some account.

The suitability of this standard for the assembly traceability use case becomes apparent when considering batches as fungible tokens with many instances each, and unique parts as non-fungible tokens with a single instance respectively. Thus, each token is the virtual representation of one or more physical parts, which are identical in regards to their traceability data. Each instance subsequently represents a single physical object or part. Each TRA is assigned a fixed account and address. Apart from the pure tokenization and management of tokens, ERC 1155 also provides functionality for transferring instances of tokens between accounts, and thus lays the foundation for tracing the location of

distributed assembly parts throughout the supply chain. While the right to operate an account's balance and transfer instances from one account to another initially resides with the account in question, ERC 1155 also enables each account to grant these rights to another via the so-called operatorAllowance mapping. The standard also suggests storing additional information about tokens off-chain, and publishing unique resource identifiers (URIs) for these files in events indexed with the respective token ID. This structure avoids computationally costly on-chain storage, while still providing verifiable, immutable linkages between tokens and meta-information.

The developed ATM extends the basic ERC 1155 by traceability-specific functionalities to cover the whole spectrum of manufacturing data beyond the pure location of parts. This requires introducing the aforementioned concept of “control”, which, in the context of the ATM, refers to the permission of an account with a specific address to process tokens and instances thereof. In Solidity, this relationship is again expressed as a double mapping data structure, as follows:

```
mapping (uint256 => mapping(address => bool)) public controllers;
```

The outer layer maps a token ID (256-bit unsigned integer) to a map of accounts (address) with control rights (bool). This allows a granular approach to distributing token handling permissions. In conjunction with the operatorAllowance included in the ERC 1155 standard, differing hierarchical structures of companies in a supply can be replicated in the traceability application. For readability purposes, both the controllers and the operatorAllowance mappings are checked in modifiers which only permit the function they modify to pass if a specified condition is met. The modifier operatorOnly(account) requires the caller of a function (msg.sender) to either be that account or listed as an operator for it. The modifier controllerOnly(token) requires the caller of a function (msg.sender) to be listed as controller of the token. The ATM follows the standards principle by storing information that is not critical to the token processing, namely serial number and status, in event logs that reference the token ID. While this data can in principle also be stored off-chain, it is preferable to explicitly store it on-chain due to its general relevance for all parts and visibility benefits. An overview of the stored data and where can be seen is given in Table 3.

The precise implementation of the token creation function is shown in the “create token” code. The ATM allows users to specify data upon token creation, such as initial supply or serial number, diverging from the reference implementation of ERC 1155 (lines 2 and 3). All information is published in events that are stored in the blocks (“emit” keyword in lines 6 and 8). The function transfers an amount of _initialSupply instances of a new token with the next available nonce to the _actor (TRA) and assigns control over the token to the operator (lines 4–7). Events logging these procedures as well as the published initial meta-information are emitted (e.g. lines 10, 12, and 13).

```
1 //create token
2 function create(uint256 _initialSupply, string calldata _uri, string calldata
3   _serialNumber, address _actor) operatorOnly(_actor) external returns(uint256 _id) {
4   _id = ++nonce;
5   controllers[_id][msg.sender] = true;
6   emit controllerUpdate("added", msg.sender, _id);
7   balances[_id][_actor] = _initialSupply;
8   emit TransferSingle(msg.sender, address(0x0), _actor, _id, _initialSupply);
9   if (bytes(_uri).length > 0)
10     emit URI(_uri, _id, _actor);
11   if (bytes(_serialNumber).length > 0)
12     emit serialNumber(_serialNumber, _id, _actor);
13     emit status("created", _id, _actor);
14 }
```

The transformation logic of assembly processes (as shown in Fig. 2) is further incorporated into the ATM. For this purpose, a new function called “craft” is introduced. The function takes an array of input token IDs, an array of input token quantities (same order as token IDs), the output quantity as well as the executing TRA and metadata information (lines 16–18). After validating that both input arrays are of the same length (lines 19–20), that the TRA has the specified quantity of each

Furthermore, update functions allow users to update the status, serialNumber, and URI of a token, by emitting a new event of the respective type, as exemplified below. Status updates allow dynamic state changes for assemblies, while av-pairs can be updated through identifications and URIs.

```

38 //token status update
39 function newStatus(uint256 _id, string calldata _status, address _actor)
40 external controllerOnly(_id){
41     if (bytes(_status).length > 0)
42         emit status(_status, _id, _actor);
43 }

```

input token (lines 22–23), and that the message sender controls the TRA-account (lines 24–25), the function “burns” these instances from the TRA’s balance and creates a new token, similar to the create function (lines 27–33). A craftedToken event is emitted, which contains the input IDs, input quantities, TRA, output quantity, and new token ID (line 36). This renders the assembly (crafting) process traceable, as input IDs and output ID are now linked.

In addition to the transactions, the emitted events are also logged in the blockchain. Each event is an element in the “log” list of a block and contains the arguments defined in the ATM, some of which might be declared as “indexed”, as well as auxiliary information. These events essentially form a database in the blockchain, which can be queried for

```

15 //craft tokens
16 function craft(uint256[] calldata _inputIds, uint256[] calldata _inputQuantities,
17 uint256 _outputInitialSupply, string calldata _uri, address _actor, string calldata
18 _serialNumber) operatorOnly(_actor) external returns(uint256 _id) {
19     require (_inputIds.length == _inputQuantities.length, "Number of input Ids
20 and Number of input quantities not equal.");
21     for (uint i = 0; i < _inputIds.length; i++) {
22         require (balances[_inputIds[i]][_actor] >= _inputQuantities[i], "Missing
23 funds for crafting");
24         require (controllers[_inputIds[i]][msg.sender] == true, "Message sender does
25 not have control over all tokens");
26     }

27     //remove (burn) instances from actor account, transfer to 0x0 as per ERC 1155
28     for (uint i = 0; i < _inputIds.length; i++) {
29         balances[_inputIds[i]][_actor] = balances[_inputIds[i]][_actor].sub
30 (_inputQuantities[i]);
31         balances[_inputIds[i]][address(0x0)] = _inputQuantities[i].add(balances
32 [_inputIds[i]][address(0x0)]);
33     }
34     emit TransferBatch(msg.sender, _actor, address(0x0), _inputIds, _inputQuantities);
35     //new token creation similar to the create function, omitted for readability
36     emit craftedToken(_inputIds, _inputQuantities, _id, _outputInitialSupply, _actor);
37 }

```

traceability relevant information via the event types and indexed arguments. In the following, a log for a *craftedToken* event is shown.

```
// event log: redundant information were omitted for clarity
address: "0x9Be12Be81e425e3d26480609d1EA2668829cBC98" //address of the ATM
blockHash: "0xc708b22ab778788adb518d3662b65fa9b9f96dd4ff2a53e83f0e20ddcfffda2a" //can be
used to verify the block
blockNumber: 88 //can be used to get timestamp
event: "craftedToken"
_actor: "0x994af70700f04d69ee15a4236dbc120BD2fb8500" //address of the TRA
_inputIds: ["84", "83", "86"] //Ids of input tokens
_inputQuantities: ["1", "4", "12"] //quantities for each input token
_outputId: "301" //Id of the output token
_outputQuantity: "1" //quantity of the output token
transactionHash: "0x63f8bc3045e2bb5df3d5d1422edb2bacd5d218635d5b1eff90ad20adf5fcd8d" //can
be used to verify the transaction
```

The ATM integrates the necessary functions for product lifecycle management, such as creating parts, maintaining parts and their relations to production resources and TRA as well as consuming parts to create new sub-modules and products following DP1 (ability to trace multi-level products) and DP2 (ability to map any input to output transformation). Additionally, different hierarchy levels of product control can be managed in the APT through *addController*, *transferControl*, and *renounceController* functions following DP3 (ability to manage part control) and DP5 (ability to manage resource associations). To virtualize product transfers, the contract inherits the ERC 1155-based *Safe(Batch)TransferFrom* and *TransferSingle* functions. As assemblies require frequent state updates to provide a holistic traceability picture, additional functions and events lay the foundation for virtualizing and tracking token states over the manufacturing flow, as defined in DP4 (ability to manage attributes and frequent state changes).

4.3. Application layer via user-friendly front-end

By default, interaction with the blockchain, such as sending messages and transactions, is only possible by using command-line interfaces as provided in Geth and API libraries like Web3.js. This constitutes a usability barrier as both methods are unintuitive for the projected userbase and lack a proper visualization of the data. Furthermore, not all traceability relevant information that end-users might be interested in is stored in the blockchain, as further av-data is stored locally. To reduce the complexity of interacting with the ATM, a higher-level interface is implemented in the form of a decentralized application. Our traceability application, called *TokenTrail*, retrieves and processes data from the blockchain database as well as off-chain data sources and then translates the data into a readable and intuitive format through a user-friendly front-end. The interaction rationale for retrieving all status events for an assembly graph is shown in Fig. 6. As off-chain data storage, we use the graph database Neo4j, which contains further traceability data, such as product, process, and resource descriptions, as well as order specifications and execution information. The interaction logic, however, applies to relational data storage analogously.

The sequence diagram displays two functions alongside the invoking arrow, where the one above tells the encapsulated function called directly by the application and the one below tells which exact web3 function or contract method is called to interact with the blockchain. Additionally, note boxes show the parameters, which are passed to these functions. The retrieving sequence starts with the user requesting traceability data for a certain part or product. Since a direct search for

the corresponding token instance of a given product UUID will loop over all existing token-product UUID pairs on the chain, long response times depending on the system scale could reduce the application's efficiency.

We, therefore, derive the token-UUID pair directly from the Neo4j database. Further data retrieval will only be initiated if the validation succeeds, otherwise, the user will be informed with a data inconsistency alert.

To display traceability-relevant information to the user, the application queries past events, which need to be aggregated from different blocks in the blockchain and displayed in their chronological sequence to the user. The timestamp of each event can be retrieved via the block number given in the event. Since the transformation records of a certain product are stored on the blockchain in different blocks, and the token instances of a product construct a graph structure, we implement the following width-first graph-traversal algorithm to retrieve all data for an assembly structure:

- 1 Find the token coupled with the given product UUID.
- 2 Get all status of the token.
- 3 Filter to the *craftedToken* event for that token.
- 4 Log input tokens for the *craftedToken* event.
- 5 Recursively implement steps 2–4 for each input token.
- 6 Finish retrieving data when reaching the bottom of the token graph.
- 7 The token graph is finally mapped to a status graph as seen below, in which each leaf layer corresponds to a stage in the product's manufacturing flow.

The resulting mapping of the product graph to the token and status graph is shown in Fig. 7. In the depicted example, token 19 consists of several child tokens with their chronological status logs, which are visualized in excerpts as a dashed line. Analogous to the product graph, in which batches may be shared by different assemblies, output tokens may link to the same input token (token 13 and 18 both integrate instances of token 11).

The application's front end then displays the chronological manufacturing status log retrieved by the algorithm by translating the identified blockchain status into manufacturing events. Additionally, the product structure of batches and unique parts is virtualized through an equivalent token structure. TRA can be identified and displayed to the user by collecting the emitted actors from each status event. The front end will be described in more detail for a specific use case and serve as a base for the solution's functional assessment.

5. Assembly use case

Traceability solutions are especially relevant for safety-critical industries, in which many companies contribute parts and sub-components to the final assembly. In the automotive industry, those rising quality and safety requirements have been identified in systematic

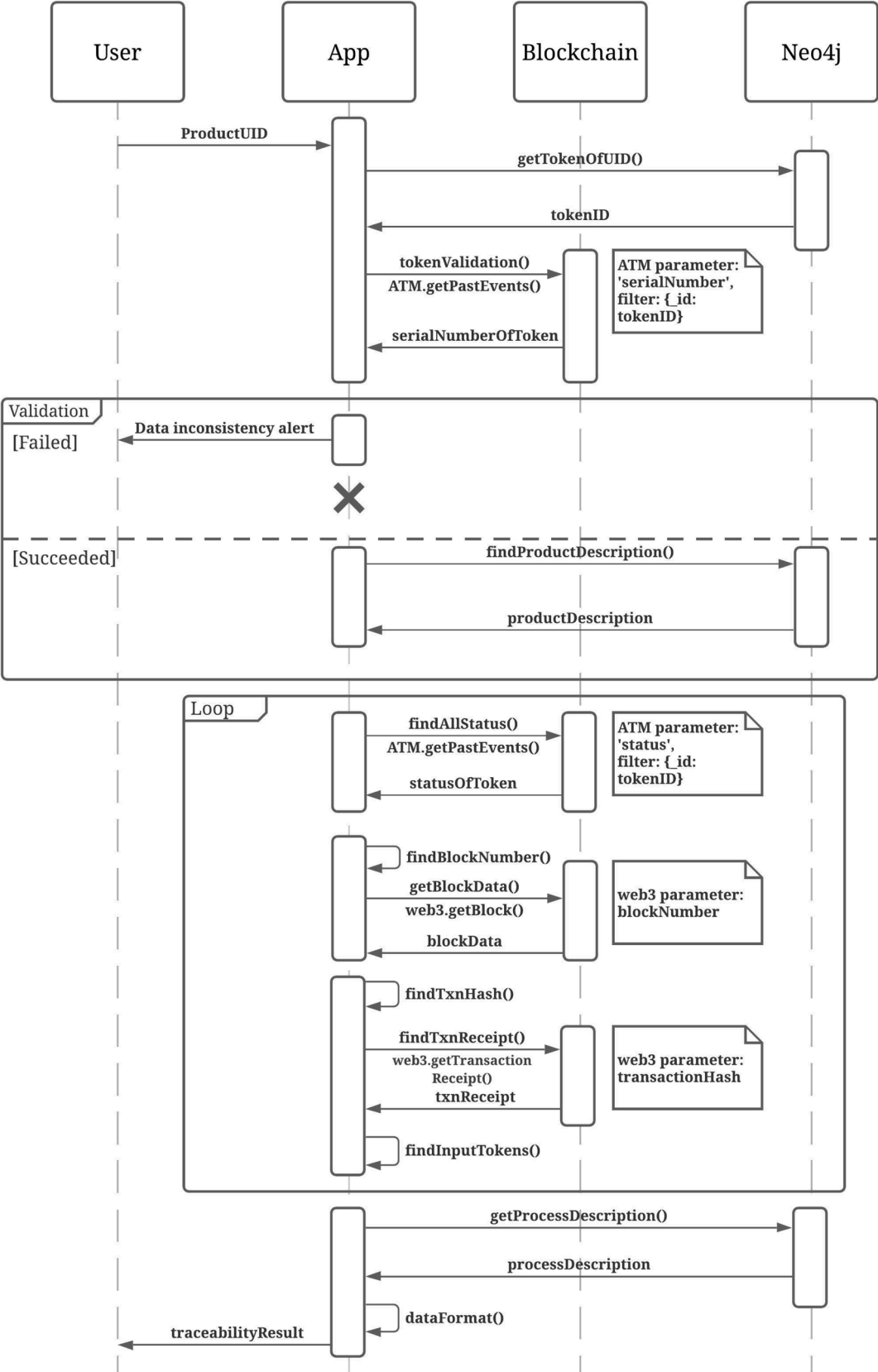


Fig. 6. Interaction architecture for decentralized application.

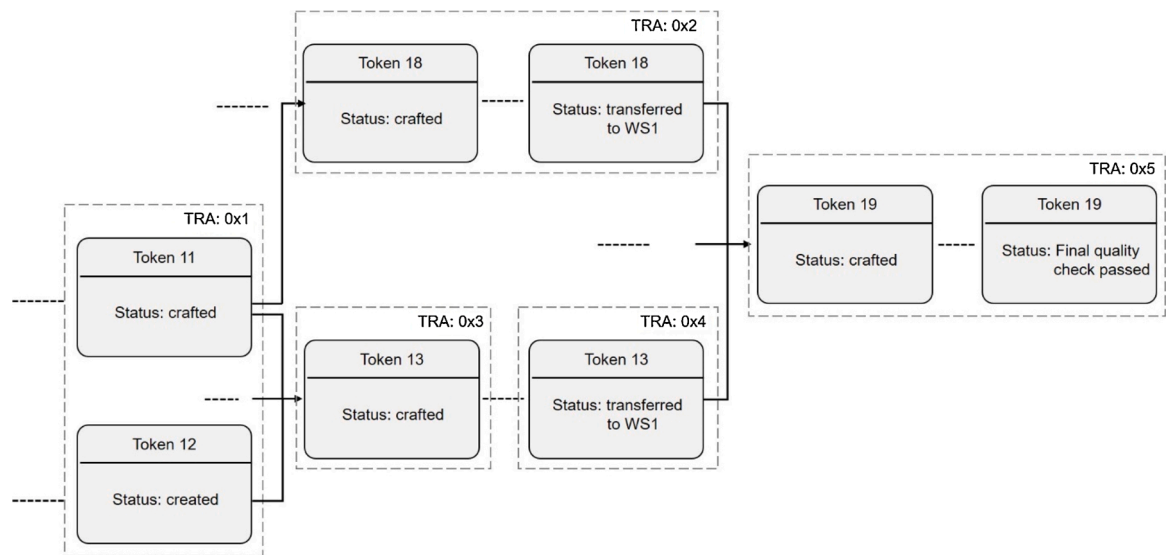


Fig. 7. Status graph of a product.

case studies and prototypes of blockchain-based solutions have been suggested [31,34]. To evaluate the functionality and performance of our solution, we, therefore, chose the assembly process of automotive electrical and electronic (E/E) systems as our testing use case. The automotive E/E system is one of the most complex products, as it consists of up to thousands of separate parts, which are assembled over various product levels using a global network encompassing several vertical supply chain stages [4,55]. The E/E system, which connects all sensors, actors, and electronic components in the vehicle, contributes to safety-critical functions, e.g. for autonomous and electrified vehicles. For European and most premium OEMs, it is produced using a make-to-order manufacturing flow so that each system is customized for its final vehicle and composes a unique part configuration consisting of

batches and unique parts [4]. The chosen use case, therefore, reflects the unique challenges of tracing complex assemblies derived in the theory section and can serve as a baseline for other products and assembly industries.

5.1. Functional evaluation

The UI of the application TokenTrail allows an evaluation of the ease of traceability for the proposed use case. The UI consists of three widgets and one main interaction interface as seen in Fig. 8. The interaction interface allows a user to specify the UIID or UBID of a traceability relevant object as input and then displays three information widgets respectively. The first widget (Info widget) shows general product

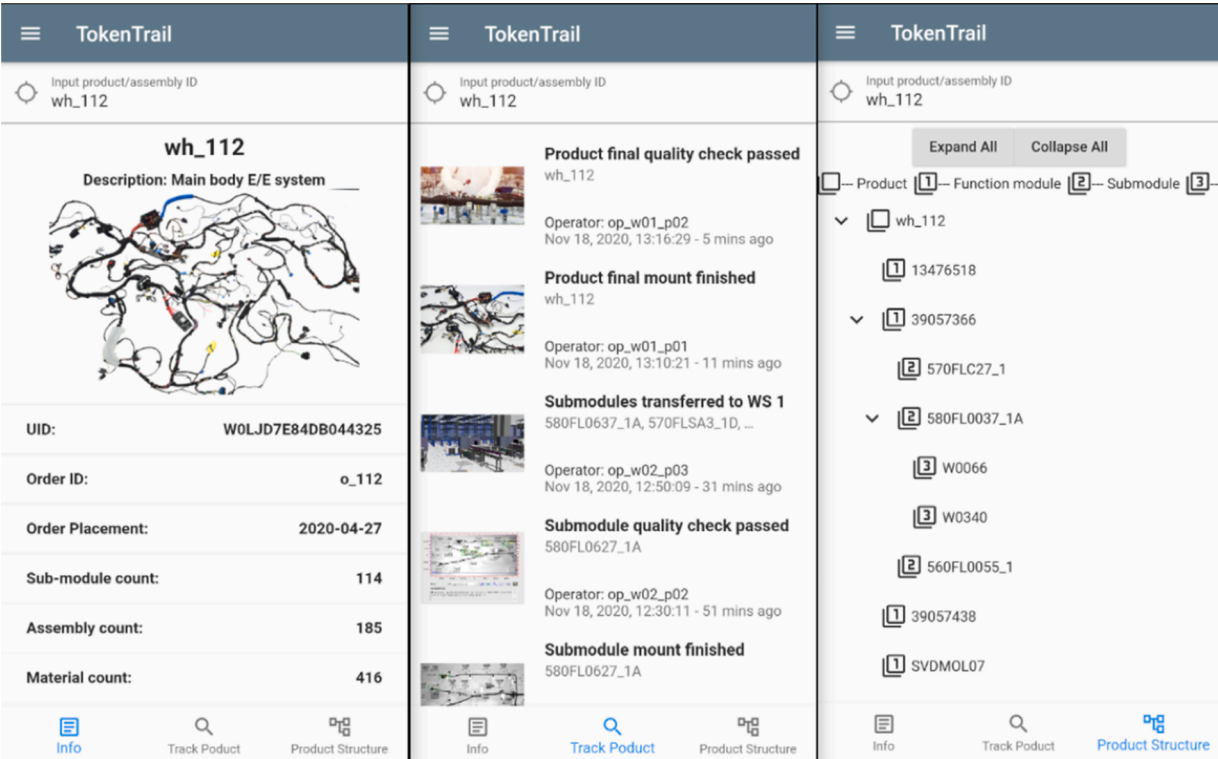


Fig. 8. Blockchain user interface for an automotive E/E system assembly traceability.

information, such as the product's UIID and an overview of its ordering data and configuration. For the automotive E/E system, the user can see the type of system (main body), the order date and identification as well as its configuration of 114 sub-modules, which in their turn consist of 185 assemblies that contain 416 batch materials or parts. The second widget displays the traceability history per filtered product reference, such as the sequence of relevant assembly and quality state events, in chronological order. For the E/E system, we can monitor assembly event logs and quality tests, as well as transfers between different TRA. Additionally, the responsible operator is logged. The dynamically changing states are listed with their timestamp of occurrence, which is derived through the described algorithms in the previous section. The third widget displays the assembly structure of the given product. While the widget "Track Product" (second view) retrieves and interprets on-chain data, the first view retrieves data from the Neo4j database and serves as a supplement to the tracking status. The UI intuitively demonstrates that our solution can provide traceability data for complex assembly structures. Each sub-module or part is virtualized as a token and can serve as a new entry point for querying its configuration and state history. The seamless integration of off-chain information and on-chain information to a holistic traceability picture mirrors actual industry environments that apply multi-purpose data management storages within their IT ecosystem. In contrast to some other proposed blockchain applications, the TokenTrail UI provides all data from a user perspective, e.g. instead of displaying the public key of a blockchain-account per status event, the UI displays the TRA or resource name. Analogously, blockchain events are not displayed in their native format but translated to manufacturing events following TS5 (intuitive user interface).

5.2. Performance metrics

When applying the ATM in a manufacturing context, the computational intensity of processing blockchain transactions and functions needs to be evaluated to identify possible performance bottlenecks. In the Ethereum network, computational consumption is measured in gas. To test the proposed solution, we configured orders and assembly BoMs for 15 E/E systems and simulated their production through a set of transformation functions. The total data volume in the case study can be determined by the number of parts and modules as well as the total number of blockchain transactions required to process the parts. The 15 configured orders consisted of 13,226 parts in total (around 700–1000 parts and module per E/E system), which resulted in approximately 15,000 blockchain transactions and 30,000 event logs. For the simulation, we used a standard laptop with an intel i7 processor 2.7 GHz and 7.7 GB RAM. In the first set-up, we configured the block time to 5 s and a gas limit to 6,000,000 gas. To evaluate the ATM's performance, we analyze the correlation between its functions and gas usage. Fig. 9 shows the overall gas usage in each order, whereby each order represents one E/E system as shown in the UI above. Every order consists of a similar set of basic materials, which are produced in large batches. When the simulation starts, there are no tokens linked to batches, so for the first order of the simulation, *o_111*, all basic parts and materials need to be created and tokens, as well as controllers, need to be attached to them, which requires a significant amount of gas. While the gas usage of these creation processes is thus accounted for in our evaluation, it might be possible to lessen its effect by distributing it across the production timeframe. Exemplarily, tokens for connector, wire, and terminal batches are created with a batch size well above 15. Starting from *o_112*, already created tokens from *o_111* can be consumed by assemblies of higher hierarchies, which reduces the needed amount of gas to create new tokens significantly.

Our simulation shows, that the most common functions in a real manufacturing environment are the crafting function, which represents the mapping and consumption of parts to assemblies of higher hierarchies as well as the safeBatchTransfer function, which virtualizes the

transaction of parts as they are connected to different resources and TRA along with the manufacturing flow. Per order or E/E system respectively, the total of all craft function calls uses an average of 7.9 million gas while transfers add up to around 5.9 million gas in our simulation. Overall, the analysis shows that tracing each product requires around 20 million gas if token creation is accounted for uniformly across all orders. Typical takt times for an automotive E/E system are in the range of 5 min so that with the calculated average gas usage, the given block time, and gas limit, we can derive that our blockchain-based traceability solution could holistically virtualize a throughput of up to 3 complex assemblies per minute. Even if we assume that our manufacturer operates 5 lines in parallel, the system would accordingly only occupy 30–33 % of its performance capacity, with 30 % assuming that most parts can be created outside of the shift, e.g. early in the morning at the goods arrival and 33 % covering the case of creating all parts during assembly. It is therefore already capable of mirroring complex assemblies of several large companies within one supply chain, whereas other automotive suppliers would probably need significantly less computational power, as their products consist of fewer parts and therefore require less creation and crafting functions. To further evaluate the system's performance potential, we kept the block time at 5 s (to ensure sufficient granularity and temporal resolution of the traceability data) but increased the gas limit to 10, 15, and 20 million gas per block respectively. Our simulation shows, that we can increase the order throughput per minute from 3 orders (6 million gas) to 5 (10 million gas), 7.5 (15 million gas), and 9.5 orders (20 million gas). This leads to an analogous decrease in performance capacity utilization from 33 % to only 11 % for the 20 million gas scenario. The analysis shows the system's capability of integrating multi-assembly products in larger supply networks and high-volume production requirements. Keeping in mind the possibilities of increasing computational power and further optimizing the code towards performance, the simulation validates the solution's general capability for a mass-customized manufacturing application. To the best of our knowledge, this is the first work, that can credibly demonstrate that it meets both the functionality and performance requirements for the presented traceability problem.

6. Discussion and conclusion

In this research, we develop a novel traceability concept for complex assembly flows. The proposed blockchain-based traceability solution is based on a PoA Ethereum network, the ERC-1155-based ATM smart contract, and the decentralized application TokenTrail. It allows companies to virtualize complex and safety-critical assembly structures directly to the blockchain. Within the ATM, companies can manage multiple fungible and non-fungible tokens representing batches and unique parts in one contract, which provides significant efficiency gains compared to previously suggested object digitization approaches. Through the ATM's general-purpose design and its unlimited instantiation options, the solution is equally suitable for simple products as well as customized multi-level assemblies. One of the resulting strengths is that the solutions can be used for all types of industries and use cases.

The inherent characteristics of the blockchain technology ensure, that safety-critical traceability data is stored immutably and transparent for all participants and democratically distributes data sovereignty across all participants of the supply chain. Different access and controller functions restrict data manipulation to the responsible TRA. The solution thus addresses the risks identified for conventional systems, such as the concentration of data sovereignty with the strongest company or the costs and data risks arising from the involvement of intermediaries. Overall, the recall risk due to lack of knowledge about the data of upstream or downstream process partners can be reduced, as all companies share the same data overview by providing a full node to the blockchain network, which makes the solution robust against single hardware node failures. Additionally, data integrity and resilience can be ensured through the application of cryptographic hashes, consensus

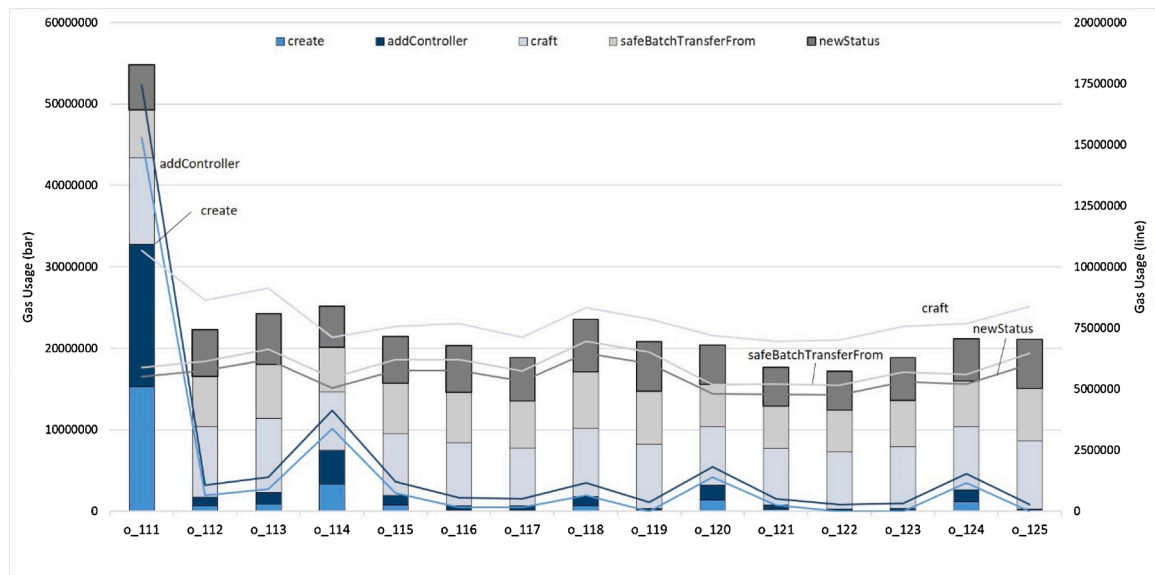


Fig. 9. Gas consumption over a simulation of 15 orders.

mechanisms, and redundant ledger storage, which together guarantee that the ledgers' data history is protected against subsequent modification or tampering attempts. Moreover, typical blockchain shortcomings concerning throughput and latency can be circumvented due to the efficient PoA consensus, the permissioned set-up, as well as the possibility of flexibly increasing block size and decreasing block time if needed.

Although the solution allows overcoming the traceability risks of centralization, there are still some limitations. Due to the open and distributed nature of the blockchain, efficient trade-offs between data protection and sufficient transparency must be found, especially as traceability data may contain intellectual property data (e.g. product composition, failure rates, etc.). It is possible to add further measures of data protection on top of the proposed solution, e.g. by storing an encrypted format of certain data entries, which is only decrypted by authorized nodes in the case of a recall. While the current prototype aims at a high degree of transparency and does not include further encryption options, it already offers a solution to link protected off-chain data to the blockchain structure through URIs. Moreover, application bugs in the smart contract pose a risk to the solution, as the smart contract is immutable once deployed to the network. This means that the smart contract needs to be carefully verified and tested before final deployment, however, updates and revisions can still be made if all participants of the network agree to it.

Functionally, the proposed decentralized traceability application TokenTrail covers all relevant aspects of a traceability system, enabling the reconstruction of part provenance, part relation exposure as well as following the product's location and processing history. The performance and scalability of the application, which is primarily expressed in the application's ability to capture manufacturing processes in real-time, was demonstrated through the automotive use case. In our simulation, the increase of gas limit per block demonstrated the solution's capability of fully virtualizing large supply networks, however, the increase in gas can not be carried on indefinitely. For the highest gas scenario, the block utilization ratio decreased to 84 % (compared to 95 % for the 6 million gas set-up). In future works, it will be necessary to find the optimal gas limit depending on the realistically needed data volume, the expected network growth, and available computer capacity in a manufacturing context. Although the simulation suggests promising results, it represents only a prototypic implementation for a limited test data set. The envisaged generalistic functionality can thus only be investigated if more use cases from different industries and sectors have

been included. In this work, we have made PPR data inherent to the blockchain solution, which, however, does not cover all data that are relevant from a traceability point of view. A detailed conception and integration of other traceability-relevant data groups, such as geospatial information or design data, would further enhance the proposed solution. In this context, the integration of different data types could be assessed with regard to the designed options, such as attribute-value pairs, URIs to off-chain data, or the interfaces to the Neo4j database.

In general, it must be said that any traceability solution, whether centralized or decentralized, must fit into an ecosystem of diverse database technologies and software systems. While a general concept of interfaces to other systems was provided, detailed assessment and comparison with other relevant traceability technologies were not conducted. The solution should thus not only be benchmarked with other blockchain technologies (e.g. assessing different clients, consensus algorithms concerning the CAP theorem, or even DLT frameworks) but also be developed in alignment with other data technologies, such as database systems (SQL, NoSQL, IPFS), data processing platforms (e.g. Apache Framework), and Internet of Things devices (e.g. smart machines). In this context, future work is required to design an interface between the ATM and computer-controlled network participants, such as manufacturing execution systems and smart machines, to enable automatically calling contract functions to store newly received information about production processes, which would allow the finished system should then be applied to and tested in a real manufacturing setting. The analytical focus during this process should be placed on evaluating the potential for the optimization of gas usage by agglomerating function calls, as well as determining scalability boundaries by comparing various combinations of block time and block gas limit. Furthermore, the general failure behavior, including cases such as gas limit exceedance, partial and complete node shutdowns, as well as manipulation attempts by internal and external attackers, should be thoroughly examined to ensure a high level of security.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Brierley JA, Cowton CJ, Drury C. A comparison of product costing practices in discrete-part and assembly manufacturing and continuous production process manufacturing. *Int J Prod Econ* 2006;100(2):314–21. <https://doi.org/10.1016/j.ijpe.2004.12.020>.
- [2] Kuhn M, Schaefer F, Otten H. Process complexity as a future challenge – a quality management perspective. *Tqm J* 2018;30(6):701–16. <https://doi.org/10.1108/TQM-10-2017-0123>.
- [3] Zhuang C, Gong J, Liu J. Digital twin-based assembly data management and process traceability for complex products. *J Manuf Syst* 2020. <https://doi.org/10.1016/j.jmsy.2020.05.011>.
- [4] Kuhn M, Franke J. Smart manufacturing traceability for automotive E/E systems enabled by event-driven microservice architecture. In: 2020 IEEE 11th International Conference on Mechanical and Intelligent Manufacturing Technologies (ICMINT); 2020. p. 142–8.
- [5] Rachuri S, Han Y-H, Foufou S, Feng SC, Roy U, Wang F, et al. A model for capturing product assembly information. *J Comput Inf Sci Eng* 2006;6(1):11–21. <https://doi.org/10.1115/1.2164451>.
- [6] Kusiak A. Service manufacturing: basic concepts and technologies. *J Manuf Syst* 2019;52:198–204. <https://doi.org/10.1016/j.jmsy.2019.07.002>.
- [7] Huang Z, Jowers C, Dehghan-Manshadi A, Dargusch MS. Smart manufacturing and DVSM based on an Ontological approach. *Comput Ind* 2020;117:103189. <https://doi.org/10.1016/j.compind.2020.103189>.
- [8] Jansen-Vullers MH, van Dorp CA, Beulens AJM. Managing traceability information in manufacture. *Int J Inf Manag* 2003;23(5):395–413. [https://doi.org/10.1016/S0268-4012\(03\)00066-5](https://doi.org/10.1016/S0268-4012(03)00066-5).
- [9] Cheng MJ, Simmons JEL. Traceability in manufacturing systems. *Int J Oper Prod Manag* 1994;14(10):4–16.
- [10] Ringsberg H. Perspectives on food traceability: a systematic literature review. *Supply Chain Manag Int J* 2014;19(5/6):558–76. <https://doi.org/10.1108/SCM-01-2014-0026>.
- [11] Agrawal R, Cheung A, Kailing K, Schonauer S. Towards traceability across sovereign, distributed RFID databases. In: 2006 10th International Database Engineering and Applications Symposium (IDEAS'06); 2006. p. 174–84.
- [12] Grundy P. Traceability's impact on manufacture. *Circuits Assembly* 2005;16(1):48.
- [13] Espinoza A, Alarcon P, Garbajosa J. Analyzing and systematizing current traceability schemas. In: 2006 30th Annual IEEE/NASA Software Engineering Workshop; 2006. p. 21–32.
- [14] Sandborn M, Olea C, White J, Williams C, Tarazaga PA, Sturm L, et al. Towards secure cyber-physical information association for parts. *J Manuf Syst* 2021;59(2):27–41. <https://doi.org/10.1016/j.jmsy.2021.01.003>.
- [15] Dupuy C, Botta-Genoulaz V, Guinet A. Batch dispersion model to optimise traceability in food industry. *J Food Eng* 2005;70(3):333–9. <https://doi.org/10.1016/j.jfoodeng.2004.05.074>.
- [16] Wu Z, Meng Z, Gray J. IoT-based techniques for online M2M-Interactive itemized data registration and offline information traceability in a digital manufacturing system. *IEEE Trans Industr Inform* 2017;13(5):2397–405. <https://doi.org/10.1109/TII.2017.2704613>.
- [17] Nakamoto S. Bitcoin: A Peer-to-Peer Electronic Cash System. [January 13, 2020]; Available from: <https://git.dhimmel.com/bitcoin-whitepaper/>.
- [18] Abeyratne Saveen A, Monfared Radmehr P. Blockchain ready manufacturing supply chain using distributed ledger. *Int J Res Eng Technol* 2016;05(09):1–10.
- [19] Bahga A, Madiseti VK. Blockchain platform for industrial internet of things. *J Softw Eng Appl* 2016;09(10):533–46. <https://doi.org/10.4236/jsea.2016.910036>.
- [20] Toyoda K, Mathiopoulos PT, Sasase I, Ohtsuki T. A novel blockchain-based product ownership management system (POMS) for anti-counterfeits in the post supply chain. *IEEE Access* 2017;5:17465–77. <https://doi.org/10.1109/ACCESS.2017.2720760>.
- [21] Hasan HR, Salah K. Blockchain-based proof of delivery of physical assets with single and multiple transporters. *IEEE Access* 2018;6:46781–93. <https://doi.org/10.1109/ACCESS.2018.2866512>.
- [22] Ali MS, Dolui K, Antonelli F. IoT data privacy via blockchains and IPFS. *The 7th International Conference on the IoT 2017*.
- [23] Tao F, Zhang Y, Cheng Y, Ren J, Wang D, Qi Q, et al. Digital twin and blockchain enhanced smart manufacturing service collaboration and management. *J Manuf Syst* 2020;37(4):517. <https://doi.org/10.1016/j.jmsy.2020.11.008>.
- [24] Huang S, Wang G, Yan Y, Fang X. Blockchain-based data management for digital twin of product. *J Manuf Syst* 2020;54:361–71. <https://doi.org/10.1016/j.jmsy.2020.01.009>.
- [25] Buterin V. A next generation smart contract & decentralized application platform: Ethereum white paper. [January 13, 2020]; Available from: http://blockchainlab.com/pdf/Ethereum_white_paper-a_next_generation_smart_contract_and_decentralized_application_platform-vitalik-buterinV.pdf.
- [26] Ethereum Foundation. Ethereum. [December 08, 2020]; Available from: <https://ethereum.org/en/>.
- [27] Ethereum developer docs. ERC-20 Token Standard. [December 03, 2020]; Available from: <https://ethereum.org/en/developers/docs/standards/tokens/erc-20/>.
- [28] Ethereum developer docs. ERC-721 Non-Fungible Token Standard. [December 03, 2020]; Available from: <https://ethereum.org/en/developers/docs/standards/tokens/erc-721/>.
- [29] Meyer T, Kuhn M, Hartmann E. Blockchain technology enabling the Physical Internet: a synergetic application framework. *Comput Ind Eng* 2019;136:5–17. <https://doi.org/10.1016/j.cie.2019.07.006>.
- [30] Hasan M, Starly B. Decentralized cloud manufacturing-as-a-service (CMaaS) platform architecture with configurable digital assets. *J Manuf Syst* 2020;56(2):157–74. <https://doi.org/10.1016/j.jmsy.2020.05.017>.
- [31] Miehle D, Henze D, Seitz A, Luckow A, Bruegge B. PartChain: a decentralized traceability application for multi-tier supply Chain networks in the automotive industry. 2019 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPCON) 2019:140–5.
- [32] Westerkamp M, Victor F, Kupper A. Blockchain-based supply Chain traceability: token recipes model manufacturing processes. 2018 IEEE International Conference on Internet of Things (Things) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData) 2018:1595–602.
- [33] Westerkamp M, Victor F, Küpper A. Tracing manufacturing processes using blockchain-based token compositions. *Digit Commun Netw* 2020;6(2):167–76. <https://doi.org/10.1016/j.dcan.2019.01.007>.
- [34] Kuhn M, Nguyen HG, Otten H, Franke J. Blockchain enabled traceability – securing process quality in manufacturing chains in the age of Autonomous driving. 2018 IEEE International Conference on Technology Management, Operations and Decisions (ICTMOD) 2018:131–6.
- [35] Kuhn M, Funk F, Franke J. Blockchain architecture for automotive traceability. *Procedia Cirp* 2021;97:390–5. <https://doi.org/10.1016/j.procir.2020.05.256>.
- [36] Linn J, Richard Hunszu L. An automatic assembly liaison extraction method and assembly liaison model. *Iie Trans* 1999;31(4):353–63. <https://doi.org/10.1080/07408179908969839>.
- [37] Matías JH, García HP, García AV. Automatic generation of a bill of materials based on attribute patterns with variant specifications in a customer-oriented environment. *J Mater Process Technol* 2008;199(1–3):431–6. <https://doi.org/10.1016/j.matprotec.2007.08.038>.
- [38] Kolisch R. Integration of assembly and fabrication for make-to-order production. *Int J Prod Econ* 2000;68(3):287–306. [https://doi.org/10.1016/S0925-5273\(99\)00011-0](https://doi.org/10.1016/S0925-5273(99)00011-0).
- [39] Kuhn M, Franke J. Data continuity and traceability in complex manufacturing systems: a graph-based modeling approach. In press. *Int J Comput Integr Manuf* 2021. <https://doi.org/10.1080/0951192X.2021.1901320>.
- [40] Biemans FP, Vissers CA. Reference model for manufacturing planning and control systems. *J Manuf Syst* 1989;8(1):35–46.
- [41] Ahmad M, Ferrer BR, Ahmad B, Vera D, Martinez Lastra JL, Harrison R. Knowledge-based PPR modelling for assembly automation. *Cirp J Manuf Sci Technol* 2018;21:33–46. <https://doi.org/10.1016/j.cirpj.2018.01.001>.
- [42] Lohse N. Towards an ontology framework for the integrated design of modular assembly systems. PhD thesis submitted to the University of Nottingham. Nottingham. 2006.
- [43] B-h Zhou, S-j Wang, L-f Xi. Data model design for manufacturing execution system. *J Manuf Technol Manag* 2005;16(8):909–35. <https://doi.org/10.1108/17410380510627889>.
- [44] Cao H, Folan P, Mascolo J, Browne J. RFID in product lifecycle management: a case in the automotive industry. *Int J Comput Integr Manuf* 2009;22(7):616–37. <https://doi.org/10.1080/09511920701522981>.
- [45] Huang GQ, Qu T, Zhang Y, Yang HD. RFID-enabled product-service system for automotive part and accessory manufacturing alliances. *Int J Prod Res* 2012;50(14):3821–40. <https://doi.org/10.1080/00207543.2011.592863>.
- [46] Jiang H, Qin S, Fu J, Zhang J, Ding G. How to model and implement connections between physical and virtual models for digital twin application. *J Manuf Syst* 2020. <https://doi.org/10.1016/j.jmsy.2020.05.012>.
- [47] Jeon BW, Um J, Yoon SC, Suk-Hwan S. An architecture design for smart manufacturing execution system. *Comput Aided Des Appl* 2017;14(4):472–85. <https://doi.org/10.1080/16864360.2016.1257189>.
- [48] Enjin. ERC-1155: Smart Contract Sample Implementation. [December 03, 2020]; Available from: <https://github.com/enjin/erc-1155>.
- [49] Stefano De Angelis Leonardo Aniello, Baldoni Roberto, Lombardi Federico, Margheri Andrea, Sassone Vladimiro. PBFT vs proof-of-authority: applying the CAP theorem to permissioned blockchain. *Italian Conference on Cyber Security* 2018.
- [50] Péter Szilágyi. EIP 225: Clique proof-of-authority consensus protocol. [December 09, 2019]; Available from: <https://eips.ethereum.org/EIPS/eip-225>.
- [51] Dinh TTA, Wang J, Chen G, Liu R, Ooi BC, Tan K-L. BLOCKBENCH: a framework for analyzing private blockchains. In: Proceedings of the 2017 ACM International Conference on Management of Data; 2017. p. 1085–100.
- [52] Ethereum Foundation. Geth Documentation. [March 29, 2021]; Available from: <https://geth.ethereum.org/docs/>.
- [53] Leal F, Chis AE, González-Vélez H. Performance evaluation of private ethereum networks. *SN Computer Science* 2020;1(5):84. <https://doi.org/10.1007/s42979-020-00289-7>.
- [54] Ethereum Classic Technical Reference Docs. Blocks: Ethereum. [March 29, 2021]; Available from: <https://ethereum-classic-guide.readthedocs.io/en/latest/docs/worlddb/database/blocks.html>.
- [55] Kuhn M, Nguyen HG. The future of harness development and manufacturing: Results from an expert case study. [August 13, 2019]; Available from: <https://www.faps.fau.de/seminare/the-future-of-harness-development-and-manufacturing/>.